INFT 3200 Group 7 Project Report

Christian Maglalang – [redacted]

Jackson Chung [redacted]

Tobias Pike [redacted]

March 25, 2026

Durham College

Table of Contents:

Introduction 2

System Overview 3

Architectural Diagram 4

Design Decisions 5

Solution Screenshots 6

- AWS ECR 6
- AWS ECS (Elastic Container Service) Cluster 7
- ECS Task Definitions & Running Tasks 8
- Load Balancing Configuration 11
- VPC & Network Configuration 13
- Database Migration (AWS RDS for PostgreSQL) 14
- Secrets Manager (Database Credentials) 14
- CloudWatch Logs or ECS Service Logs 15
- Final Application in Action 16

Introduction

The Reading Nook is a medium sized bookstore and café chain with locations spread across the Greater Toronto and Hamilton Area. As part of their efforts to modernize their technological stack, we have been contracted to migrate their existing inventory website into the cloud.

The organization's current inventory website consists of a Next.js frontend, a flask-based REST API, and a PostgreSQL database for persistent storage. These services are already deployed using containers however, for this project, the Reading Nook website team ask that we develop and implement a GitHub Actions CI/CD pipeline to build, store, and deploy each service to AWS incorporating ECR whenever a source commit occurs.

System Overview

The Reading Nook incorporates three primary components to create a functioning web application:

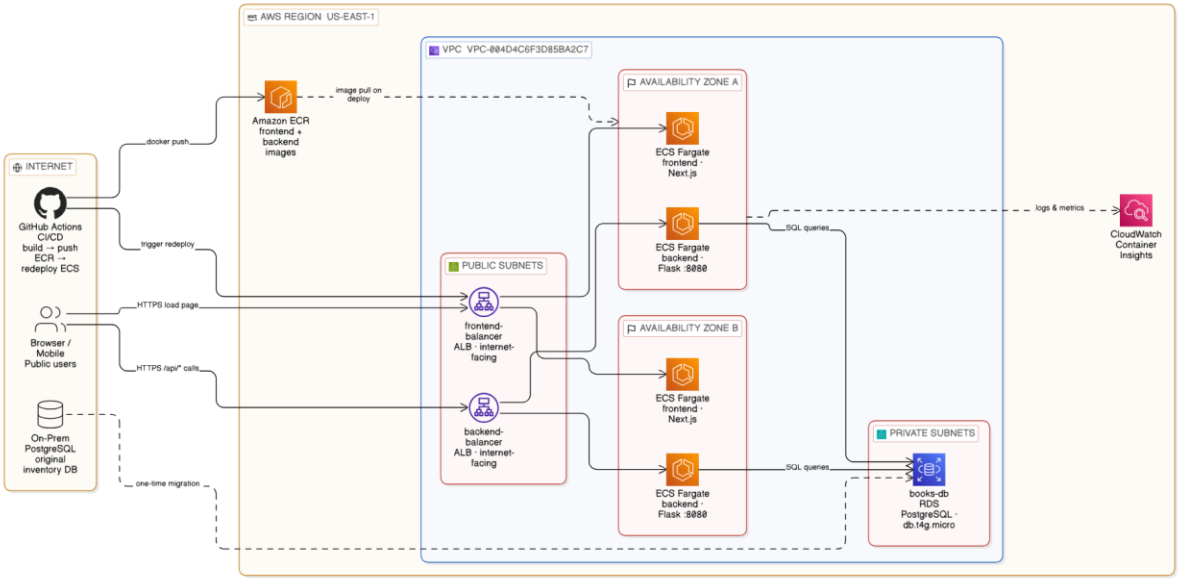
- Frontend: The user interface developed using Next.js for users to view and interact with inventory data.
- Backend: Implements Flask-based REST API to process client request and retrieve, add, update, and delete book records.
- Database: Data is handled by a PostgresSQL database hosted on Amazon RDS, storing information on book inventory.

Application components are containerized using Docker and deployed using Amazon Elastic Container Service (Amazon ECS). Container images are stored in Amazon Elastic Container Registry (Amazon ECR), allowing versioning and deployment. An Application Load

Balancer is used to distribute incoming traffic to the appropriate services, ensuring high availability and scalability.

A CI/CD pipeline is implemented using GitHub Actions. This pipeline automatically builds Docker images, pushes them to ECR, and updates the running services in ECS whenever changes are committed to the source repositories.

Architectural Diagram



Design Decisions

Amazon ECS with Fargate was used to run the containerized services without needing to deal with servers directly. This cuts down on setup and maintenance and also lets the frontend (Next.js) and backend (Flask API) run as separate services so they can be deployed and scaled independently.

Amazon ECR was used to store Docker images for both services. Keeping everything inside AWS makes deployments faster and avoids having to deal with external registries or extra authentication setup.

A CI/CD pipeline was built using GitHub Actions. Every time code is pushed, new Docker images are built, pushed to ECR, and deployed to ECS using rolling updates. This keeps deployments consistent and avoids having to manually update anything.

Application Load Balancers were set up to handle incoming traffic and route it to the correct containers. They also run health checks, so if something goes down, traffic is automatically redirected without needing manual intervention.

The system is deployed across multiple availability zones to improve reliability. Right now, the services are running with a low task count, but the setup supports scaling across zones if needed for better redundancy.

Amazon RDS for PostgreSQL was used for the database. It handles things like backups, patching, and general maintenance, which saves time, and it's placed in private subnets, so it isn't directly exposed to the internet.

AWS Secrets Manager is used to store database credentials instead of hardcoding them. This makes things more secure and allows credentials to be changed without rebuilding or redeploying the app.

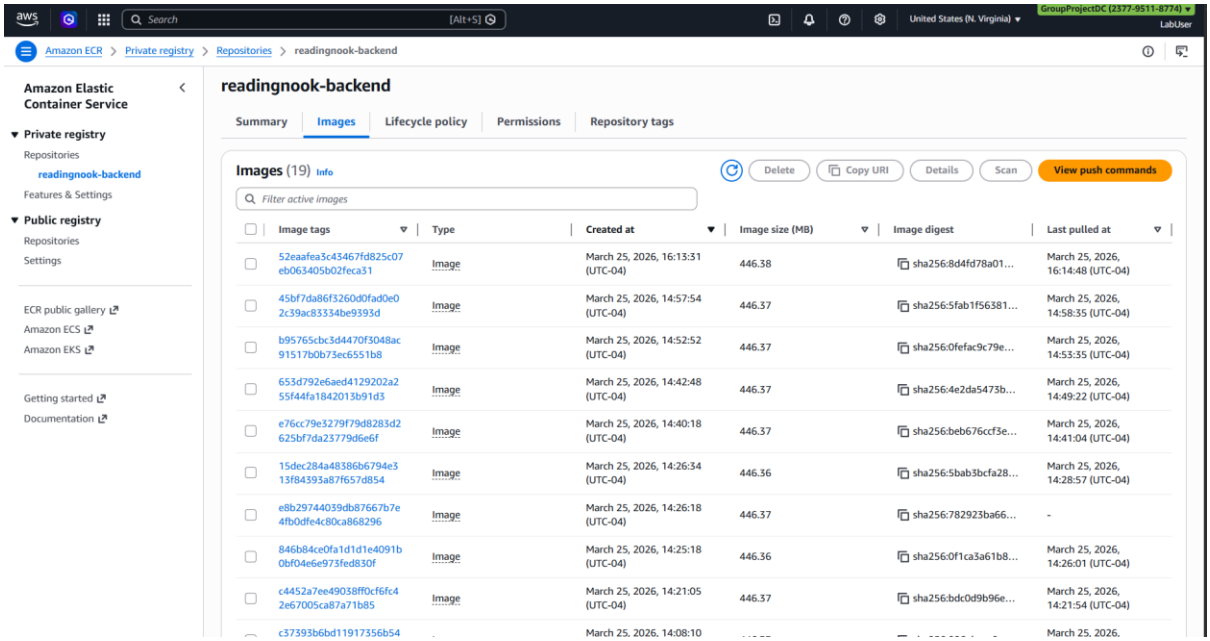
Amazon CloudWatch with Container Insights is used for logging and monitoring. It gives visibility into how the services are running and makes it easier to catch and debug issues.

Solution Screenshots

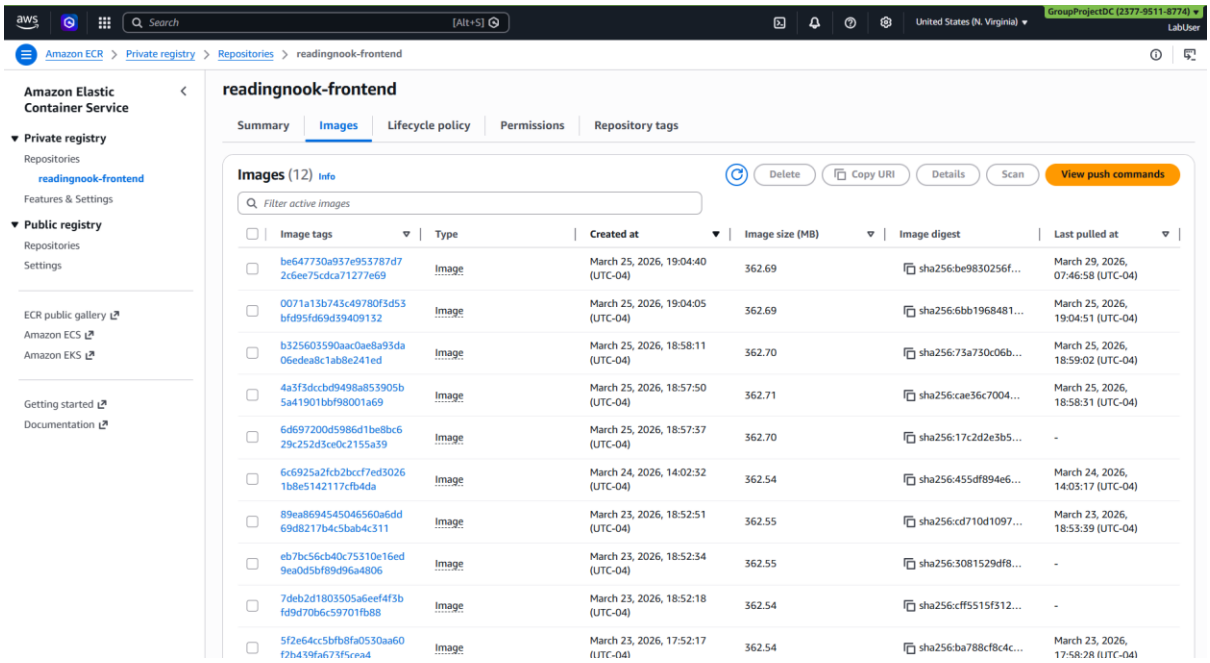
AWS ECR

The repository page in ECR, showing the successfully pushed Docker images for each microservice:

Backend Repository:

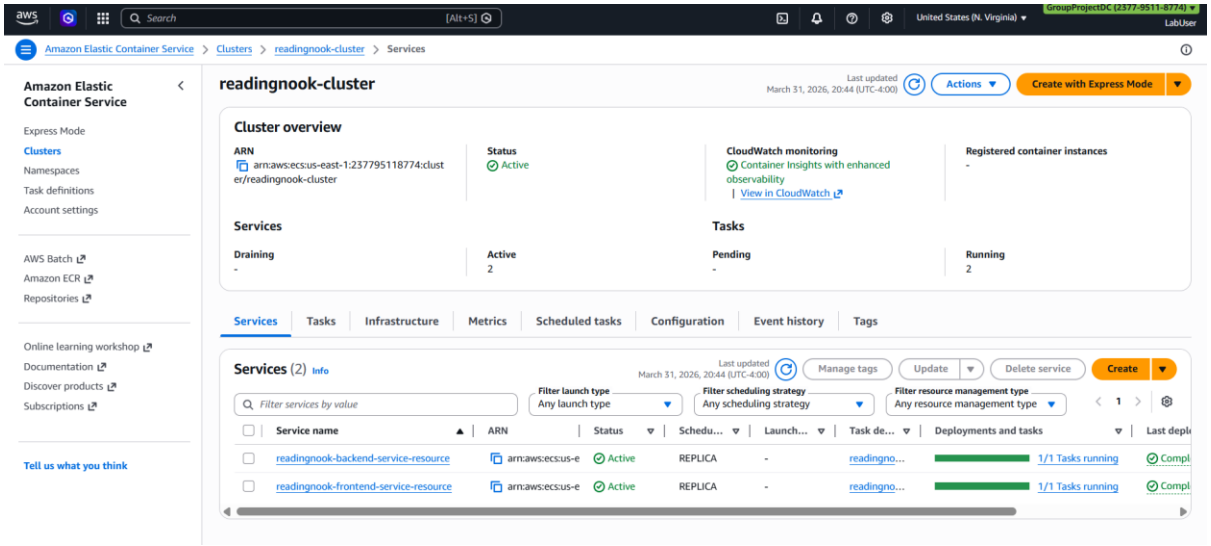


Frontend Repository:



AWS ECS (Elastic Container Service) Cluster

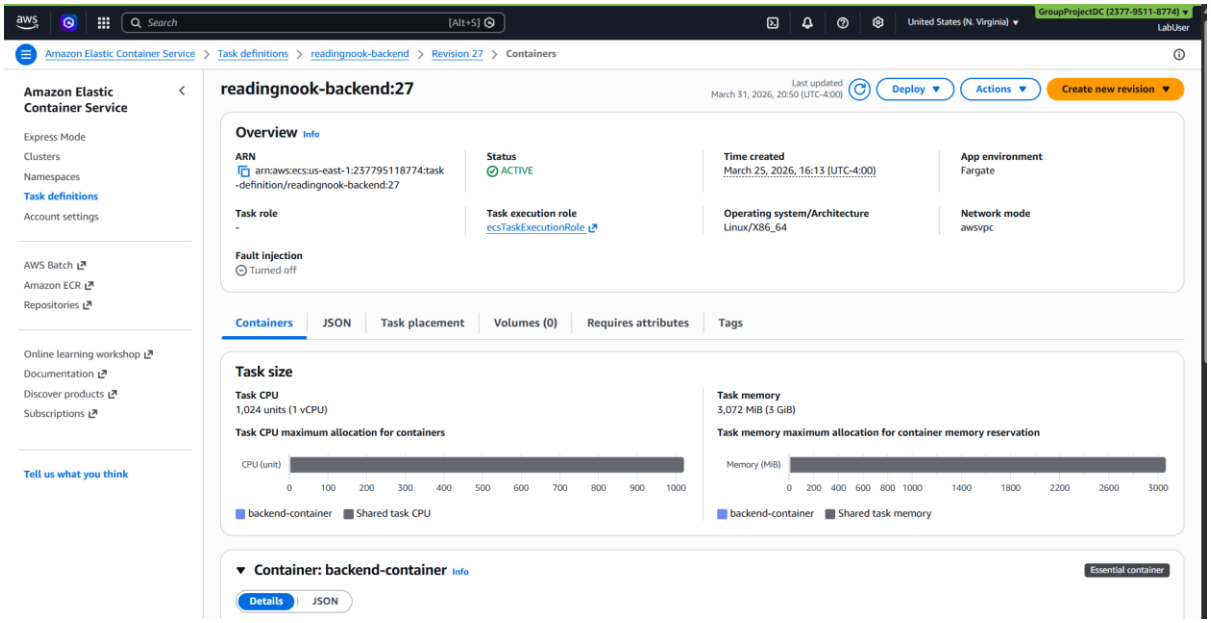
ECS Console with your cluster visible, indicating the active tasks/services:

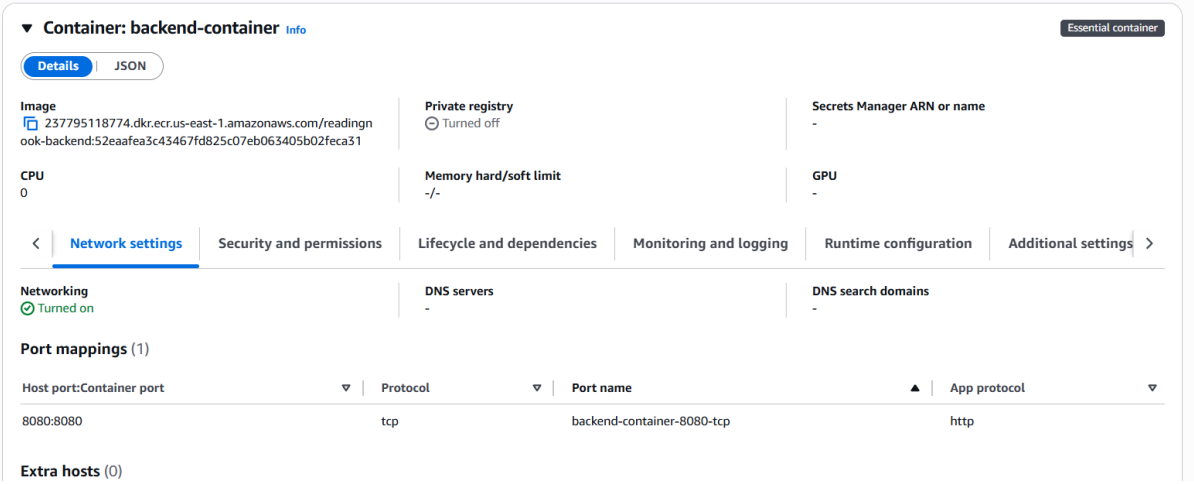


ECS Task Definitions & Running Tasks

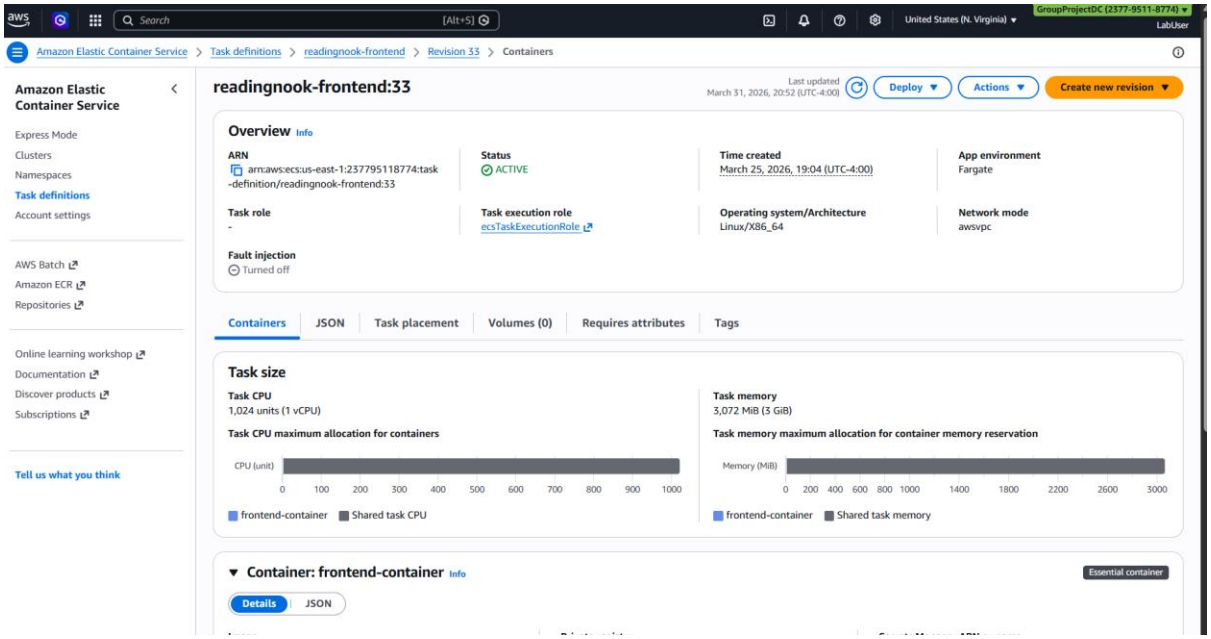
Task Definitions tab displaying the definitions for each microservice (CPU/memory requirements, container port mappings, etc.).

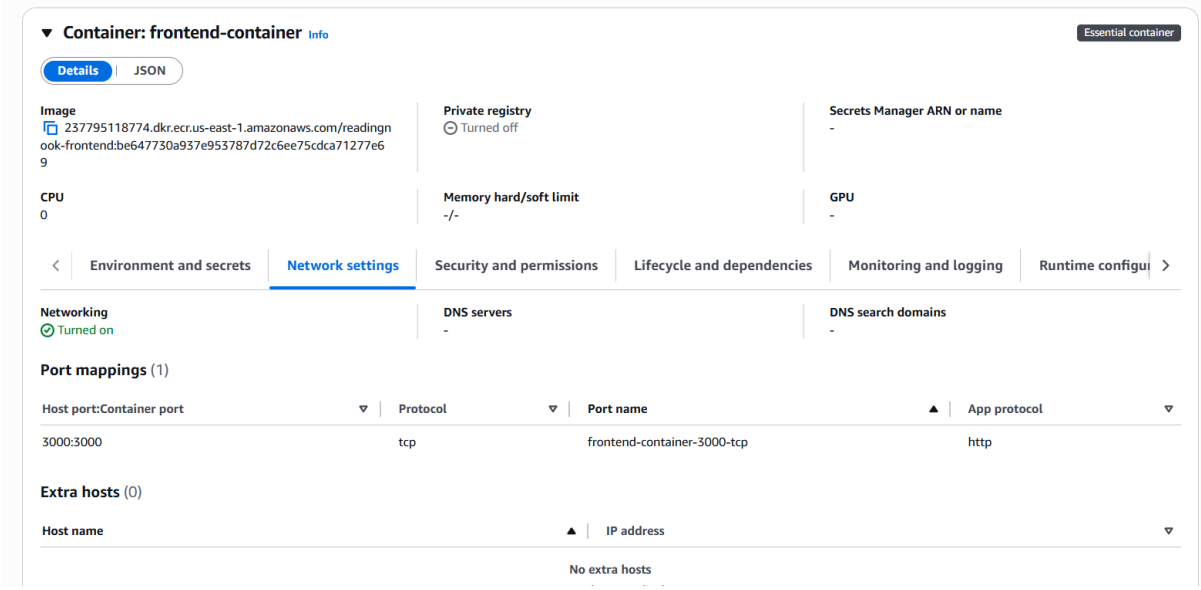
Backend:


Amazon Elastic Container Service > Task definitions > readingnook-backend > Revision 27 > Containers
Amazon Elastic Container Service
Express Mode
Clusters
Namespaces
Task definitions
Account settings
AWS Batch
Amazon ECR
Repositories
Online learning workshop
Documentation
Discover products
Subscriptions
Tell us what you think
readingnook-backend:27
Last updated March 31, 2026, 20:50 (UTC-4:00)
Deploy
Actions
Create new revision
Overview Info
ARN
arn:aws:ecs:us-east-1:237795118774:task-definition/readingnook-backend:27
Status
ACTIVE
Time created
March 25, 2026, 16:13 (UTC-4:00)
App environment
Fargate
Task role
-
Task execution role
ecsTaskExecutionRole
Operating system/Architecture
Linux/X86_64
Network mode
awsvpc
Fault injection
Turned off
Containers
JSON
Task placement
Volumes (0)
Requires attributes
Tags
Task size
Task CPU
1,024 units (1 vCPU)
Task memory
3,072 MiB (3 GiB)
Task CPU maximum allocation for containers
Task memory maximum allocation for container memory reservation
CPU (unit)
Memory (MiB)
backend-container
Shared task CPU
Shared task memory
Container: backend-container Info
Essential container
Details
JSON

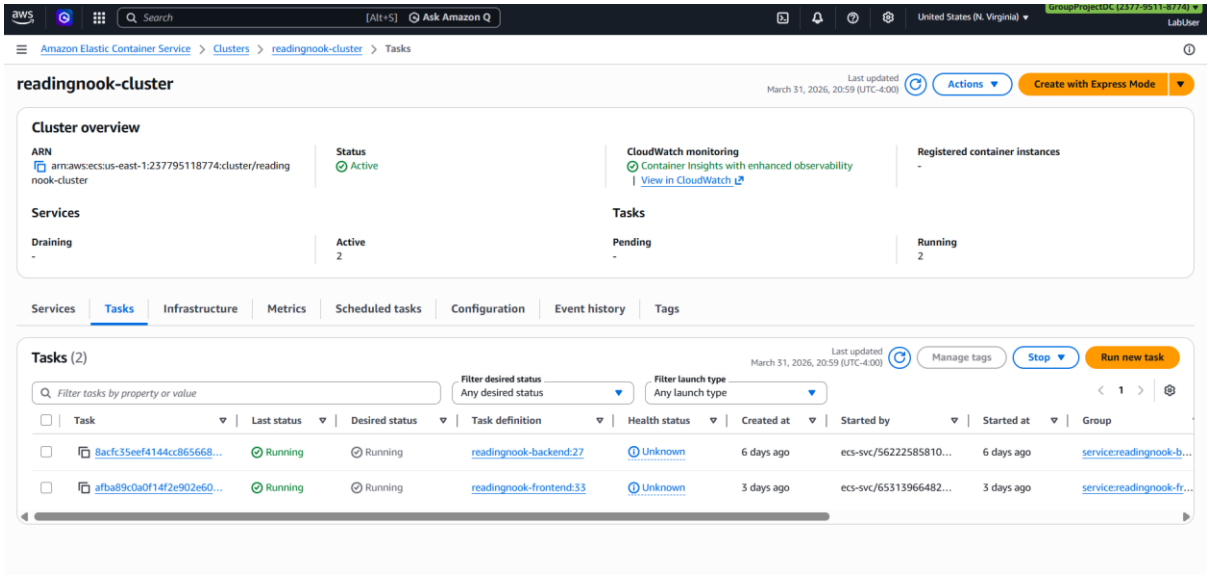

Container: backend-container Info
Essential container
Details
JSON
Image
237795118774.dkr.ecr.us-east-1.amazonaws.com/readingnook-backend:52eaafea3c43467fd825c07eb063405b02feca31
Private registry
Turned off
Secrets Manager ARN or name
-
CPU
0
Memory hard/soft limit
-/-
GPU
-
Network settings
Security and permissions
Lifecycle and dependencies
Monitoring and logging
Runtime configuration
Additional settings
Networking
Turned on
DNS servers
-
DNS search domains
-
Port mappings (1)
Host port:Container port
Protocol
Port name
App protocol
8080:8080
tcp
backend-container-8080-tcp
http
Extra hosts (0)

Frontend:





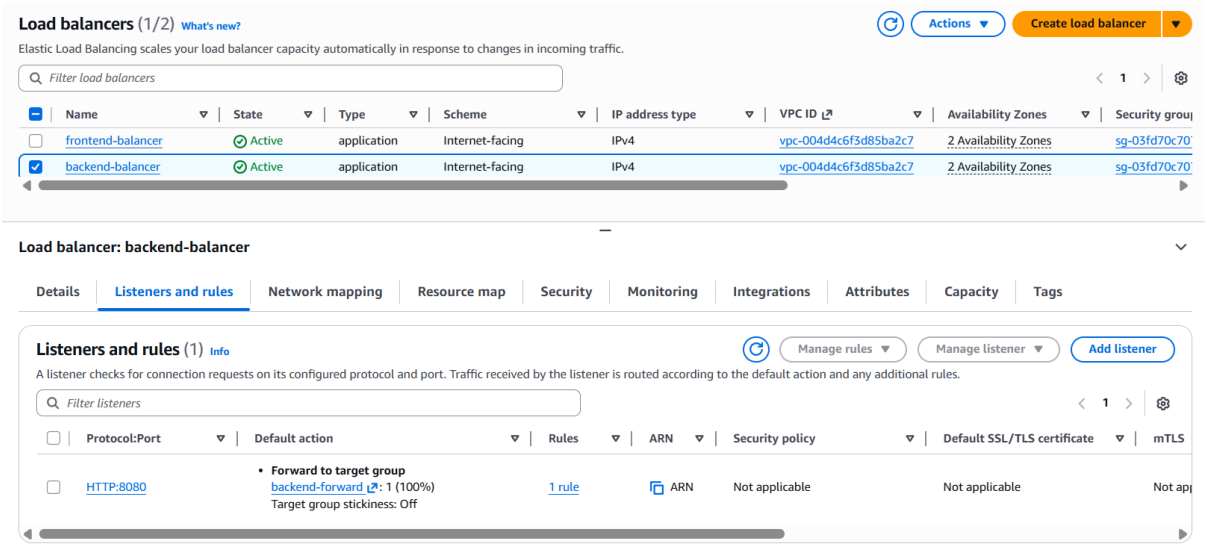
Running Tasks tab showing healthy tasks for each service.



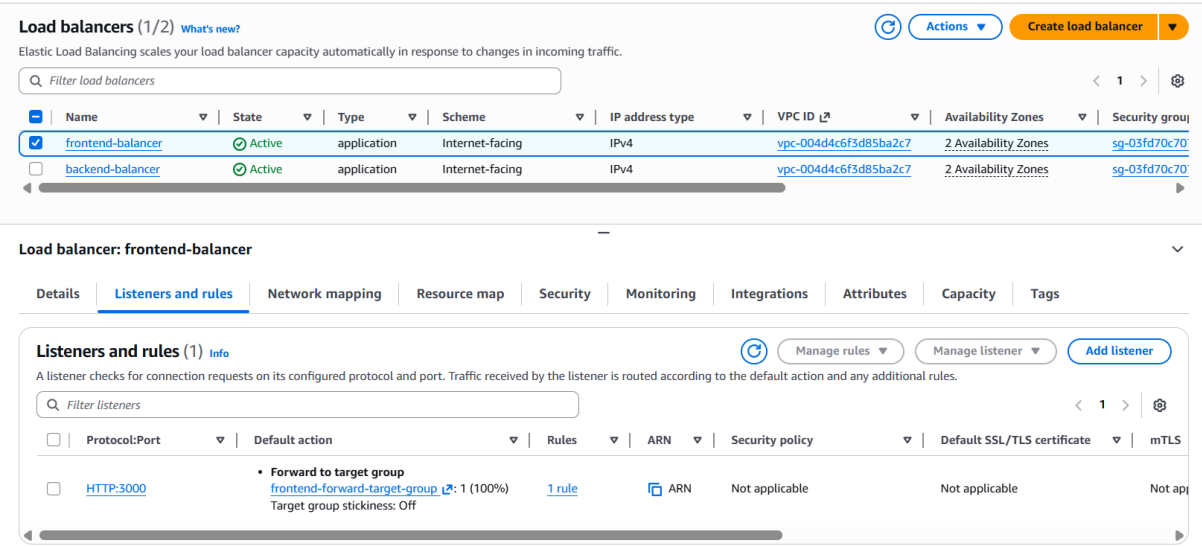
Load Balancing Configuration

Load Balancer in the Amazon or ECS console showing listener rules.

Backend:

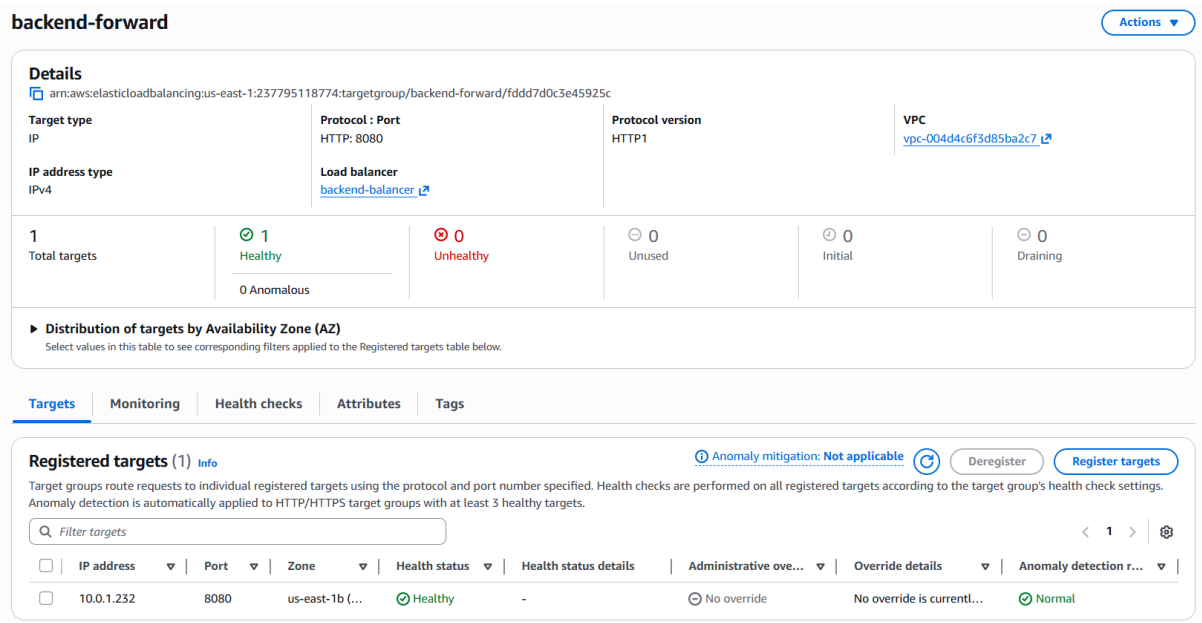


Frontend:

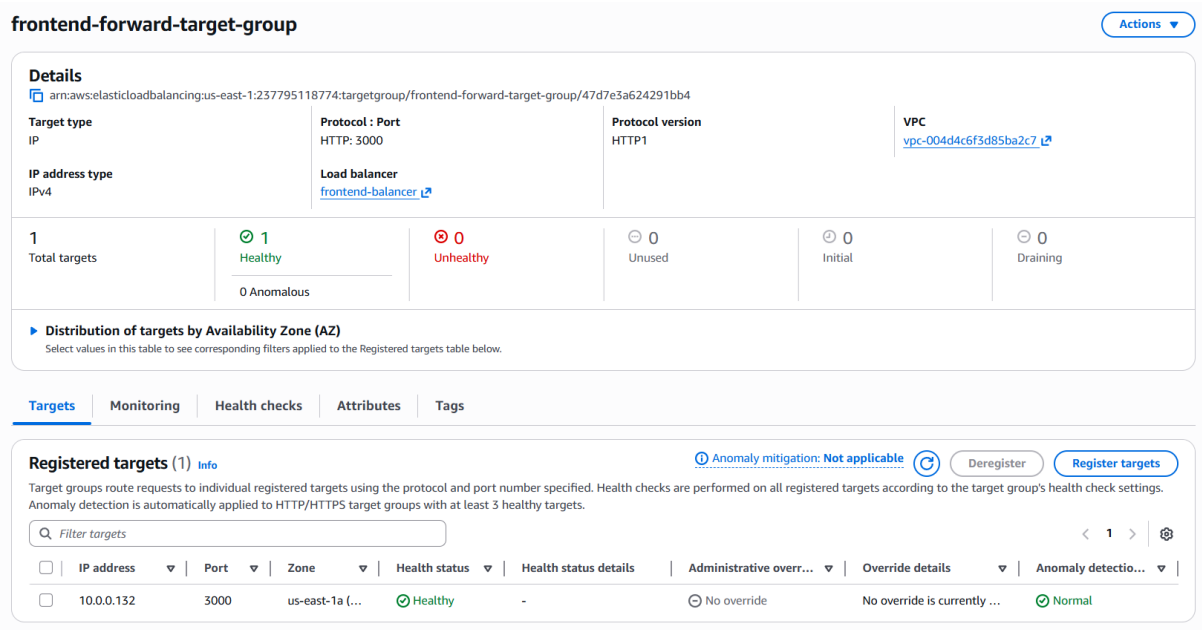


Target Groups page showing how traffic routes to the correct container tasks:

Backend:

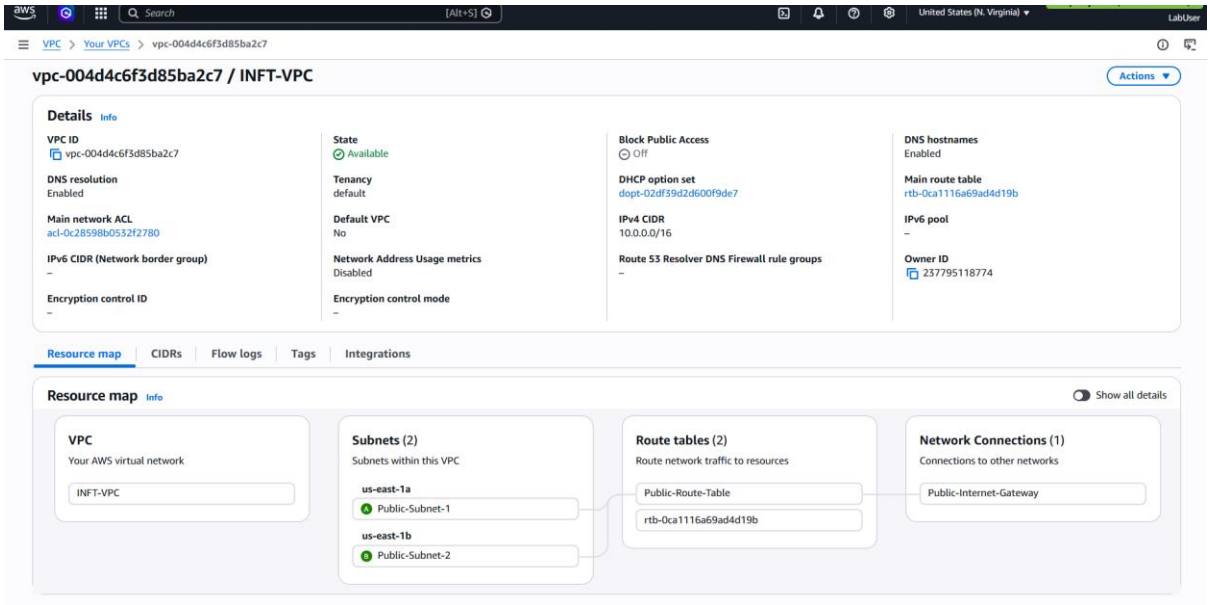


Frontend:



VPC & Network Configuration

VPC settings indicating subnets, route tables, and security groups:



Security Group rules specifically allowing traffic from the load balancer to container tasks or database:

sg-03fd70c70739b622f - inft-stack-vpc-WebSecurityGroup-s1tca10kMJlv

sg-03fd70c70739b622f - inft-stack-vpc-WebSecurityGroup-s1tca10kMJlv

Actions

Details

Security group name	Security group ID	Description	VPC ID
inft-stack-vpc-WebSecurityGroup-s1tca10kMJlv	sg-03fd70c70739b622f	Allows http to instance	vpc-004d4c6f3d85ba2c7

Owner	Inbound rules count	Outbound rules count
237795118774	7 Permission entries	2 Permission entries

Inbound rules | Outbound rules | Sharing | VPC associations | Related resources - *new* | Tags

Inbound rules (7)

Manage tags | Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
–	sgr-0c33b8505dc738806	IPv4	HTTP	TCP	80	0.0.0.0/0
–	sgr-03a6db8fbe5f52700	IPv6	All traffic	All	All	::/0
–	sgr-0223f1e2f9a2df1dd	IPv4	Custom TCP	TCP	3000	0.0.0.0/0
–	sgr-0ae154bde9d581342	IPv4	Custom TCP	TCP	8080	0.0.0.0/0
–	sgr-0b700e59a1e416211	IPv4	MYSQL/Aurora	TCP	3306	0.0.0.0/0
–	sgr-01a4f74873ef64834	IPv4	All traffic	All	All	0.0.0.0/0
–	sgr-08f62d0f1e6036514	IPv4	SSH	TCP	22	0.0.0.0/0

Outbound rules (2)

Manage tags | Edit outbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range	Destination	Description
–	sgr-0f96994048cf5e323	–	All traffic	All	All	sg-03fd70c70739b622f...	–
–	sgr-0d6e24c769889ad60	IPv4	All traffic	All	All	0.0.0.0/0	–

Database Migration (AWS RDS for PostgreSQL)

RDS console showing your PostgreSQL instance status.

aws | Search [Alt+S] | United States (N. Virginia) | LabUser

Aurora and RDS > Databases > readingnook

readingnook

Modify | Actions

Summary

DB identifier	Status	Role	Engine	Recommendations
readingnook	Available	Instance	PostgreSQL	6 Informational
CPU	**Class**	**Current activity**	**Region & AZ**	
3.84%	db.t3.micro	0 Connections	us-east-1a	

Connectivity & security | Monitoring | Logs & events | Configuration | Zero-ETL integrations | Maintenance & backups | Data migrations | Tags | Recommendations

Connect using | Info

- Code snippets — Use when connecting through SDK, APIs, or third-party tools including agents.
- CloudShell — Use for a quick access to AWS CLI that launches directly from the AWS Management Console.
- Endpoints — Use when connecting through any IDE interface.

Database name	Master username	Internet access gateway	Port
booksdb	appuser	Disabled	5432

Endpoint type: Instance endpoint

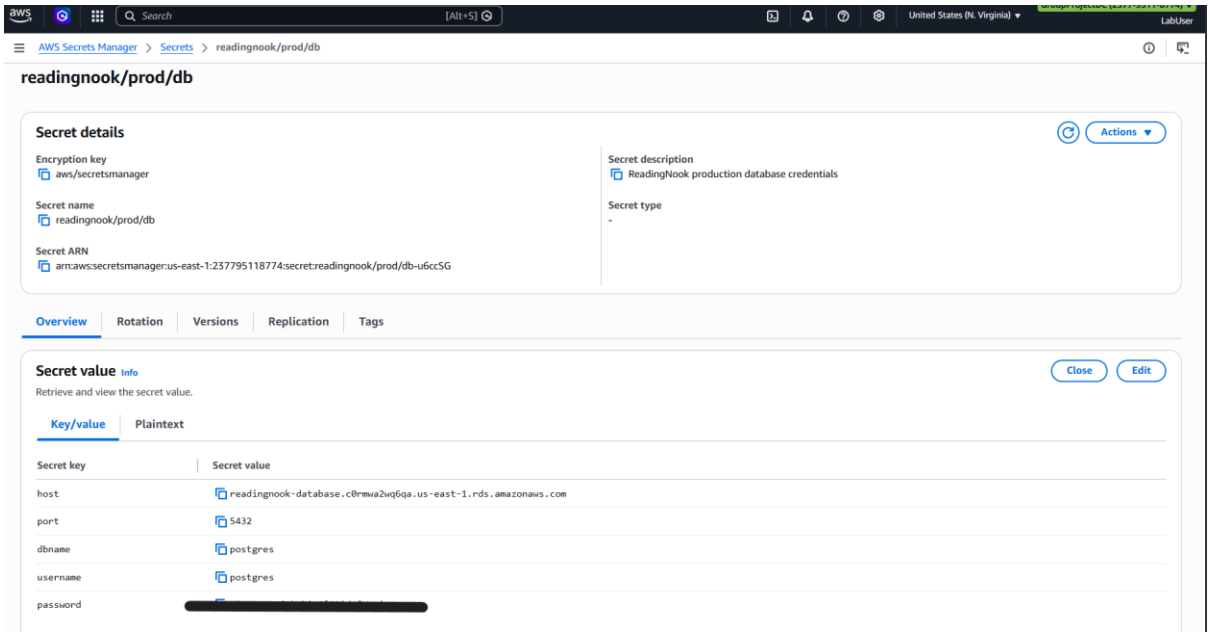
▾ Additional configurations

Connectivity & security

Endpoint & port	Networking	Security
Endpoint: readingnook.c0rmwa2wq6qa.us-east-1.rds.amazonaws.com	**Availability Zone**: us-east-1a	**VPC security groups**: inft-stack-vpc-WebSecurityGroup-s1tca10kMJlv (sg-03fd70c70739b622f) Active
Port	**VPC**: INFT-VPC (vpc-004d4c6f3d85ba2c7)	

Secrets Manager (Database Credentials)

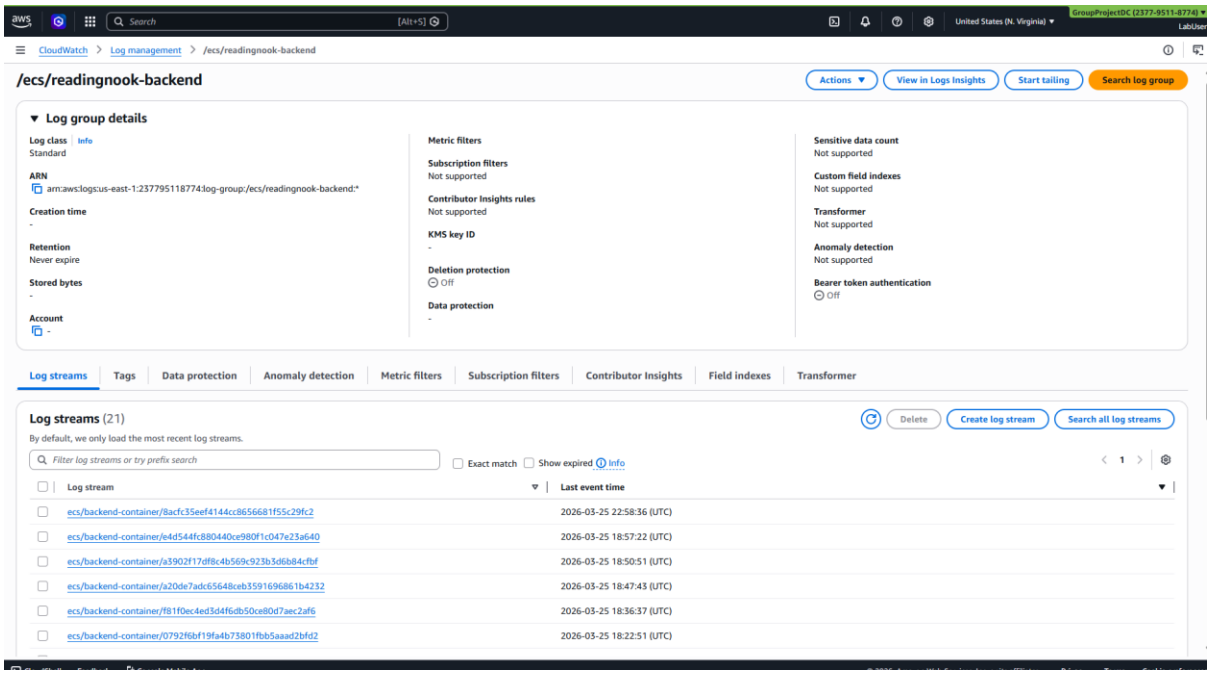
AWS Secrets Manager entry showing the stored connection details (blur out sensitive info).



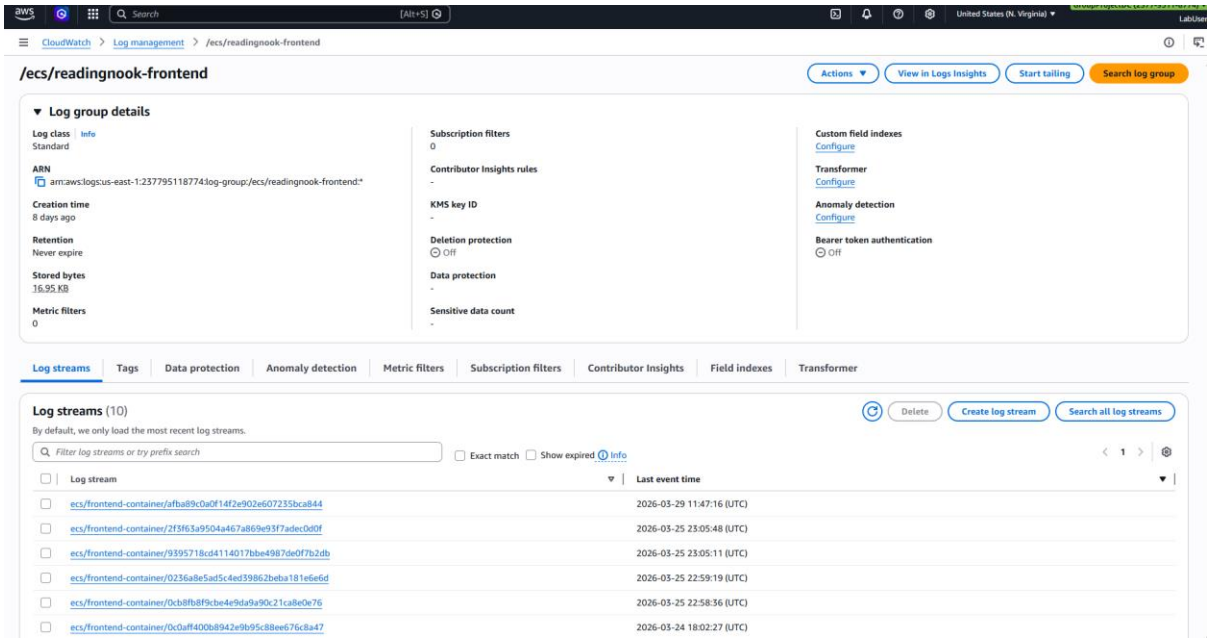
CloudWatch Logs or ECS Service Logs

CloudWatch log groups for your ECS services showing application logs or container logs.

Backend:



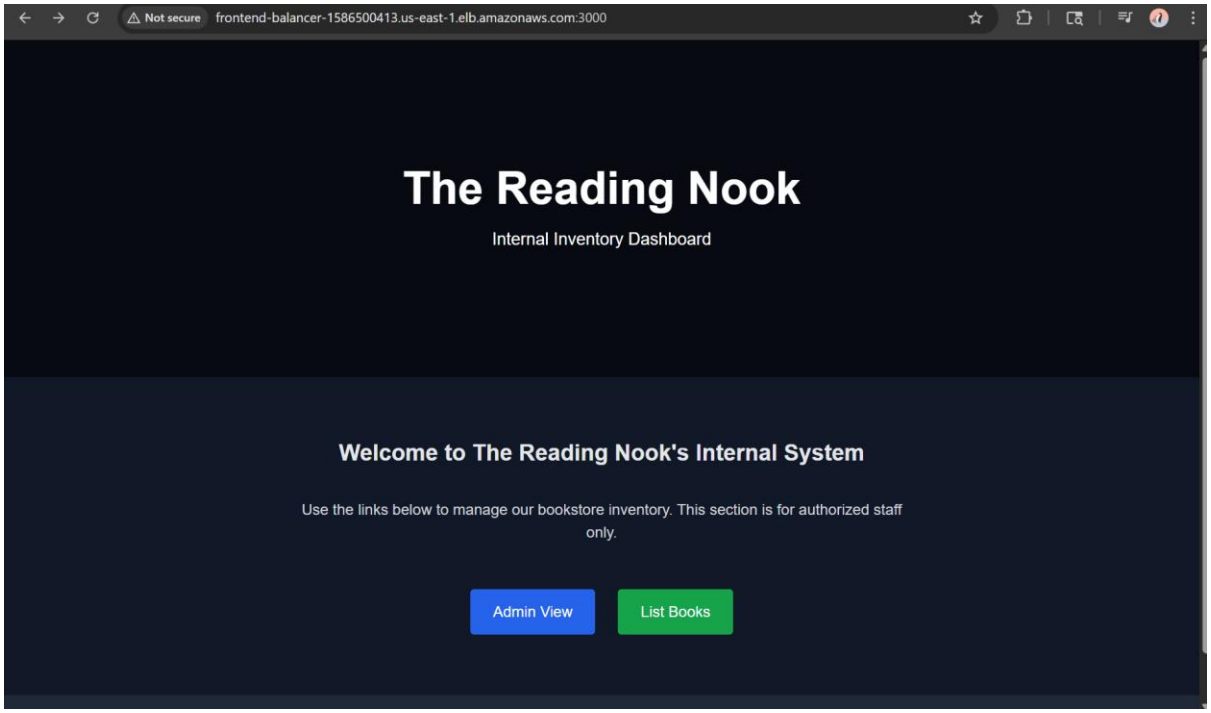
Frontend:



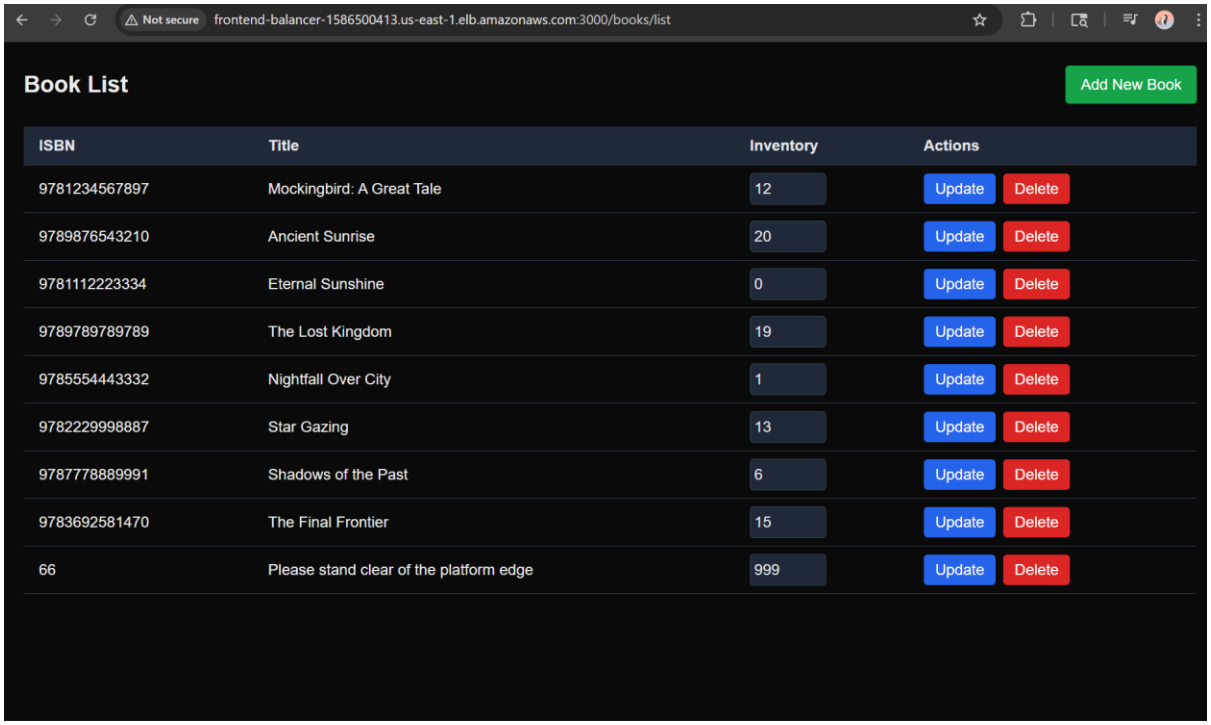
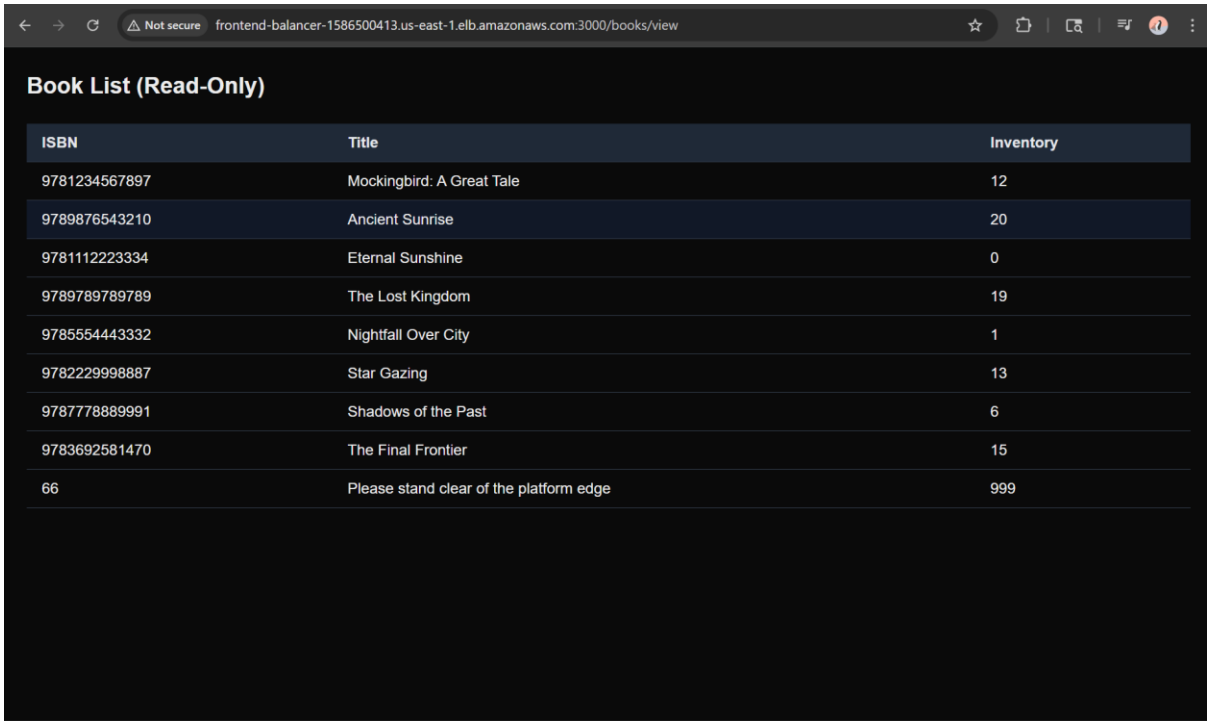
Final Application in Action

Browser window showing the Next.js inventory site running, with inventory data displayed.

Home Page:



Admin View Page:



View Books Page: